PUC CAMPINAS
PONTIFÍCIA UNIVERSIDADE CATÓLICA

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Fundamentos de Sistemas Operacionais
2s/2026 – Prof. Lucas Reis

Lista de Exercícios 1 – Preparação para P1

1 Introdução e Conceitos

1. O que é um Sistema Operacional e quais são suas principais funções?
2. O que é uma chamada de sistema (*system call*) em um sistema operacional? Por que sua existência é importante?
3. Diferencie chamadas de sistema (*system calls*) de chamadas de função comuns em termos de privilégios e transição de modo.

2 Processos e Threads

4. Descreva o ciclo de vida de um processo, detalhando seus possíveis estados.
5. O que é o Bloco de Controle de Processo (PCB) e quais informações ele armazena?
6. Explique a diferença fundamental entre processos e threads no que diz respeito ao compartilhamento de memória.
7. Considere o seguinte código em C:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int contador = 0;
    fork();
    fork();
    contador++;
    printf("Contador: %d\n", contador);
    return 0;
}
```

a) Qual será a saída impressa pelo programa? Justifique sua resposta.

b) Desenhe a árvore de processos criada a partir do primeiro processo pai e indique o valor de `contador` em cada processo.

8. Considere o seguinte código em C:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid = fork();
    if (pid == 0) {
      fork();
      printf("Hello, World!\n");
    }
    fork();
    printf("Hello, World!\n");
    return 0;
}
```

a) Qual será a saída impressa pelo programa? Justifique sua resposta.

b) Desenhe a árvore de processos criada a partir do primeiro processo pai. Assinale com um ponto dentro de cada processo toda vez que ele imprimir `"Hello, World!"`.

9. É vantajoso utilizar múltiplas threads em um processador de apenas um núcleo (*single-core*)? Justifique abordando o conceito de responsividade e I/O.

3 Escalonamento de Processos

10. Elabore sobre o trabalho do escalonador de processos e o custo da troca de contexto (*context switch*).

11. Diferencie escalonamento preemptivo de escalonamento não-preemptivo.

12. No algoritmo *Round Robin* (RR), como a escolha do tamanho do *quantum* de tempo afeta o equilíbrio entre a sobrecarga do sistema e o tempo de resposta?

13. O que é o fenômeno de *Starvation* (inanição) e por que algoritmos como SJF são suscetíveis a ele?

14. Considere os seguintes processos, com seus respectivos tempos de pico de CPU e prioridades (um número menor indica maior prioridade). Todos chegaram no tempo 0:

Processo	Pico de CPU	Prioridade
P1	6	2
P2	8	1
P3	7	3
P4	3	4

a) Desenhe o diagrama de Gantt e o tempo médio de espera para SJF.

b) Desenhe o diagrama de Gantt e o tempo médio de espera para prioridade.

c) Desenhe o diagrama de Gantt e o tempo médio de espera para Round-Robin (quantum 4).

4 Sincronização

15. Defina o conceito de Seção Crítica e explique os três requisitos fundamentais para sua solução.

16. Qual a diferença fundamental entre o uso de um Mutex e de um Semáforo Contador?

17. Descreva um exemplo prático de uma Condição de Corrida (*Race Condition*).

18. Observe o código em C abaixo:

```c
int saldo = 0;
void* depositar(void* arg) {
    int valor = *((int*)arg);
    saldo = saldo + valor;
    printf("Deposito realizado!\n");
    return NULL;
}
```

Considerando execução concorrente:

a) identifique a região da seção crítica dessa função;

b) explique qual condição de corrida pode ocorrer;

c) mostre onde inserir as operações `lock` e `unlock` de um mutex para corrigir o problema.

19. Cada chamada de `comprar` é executada por uma thread diferente:

```c
int estoque = 5;
int vendas = 0;

void* comprar(void* arg) {
    int quantidade = *((int*)arg);

    if (estoque >= quantidade) {
        estoque = estoque - quantidade;
        vendas = vendas + quantidade;
    }
    return NULL;
}
```

Duas compras podem ser aprovadas usando o mesmo valor de `estoque`. Identifique a seção crítica e mostre como usar um mutex para que o teste e as duas atualizações sejam realizados como uma única operação.

20. A thread `enviar` só pode executar `enviar_relatorio()` depois que a thread `preparar` terminar `preparar_relatorio()`:

```
void* preparar(void* arg) {
    preparar_relatorio();
    return NULL;
}

void* enviar(void* arg) {
    enviar_relatorio();
    return NULL;
}
```

Explique como usar um semáforo para impor essa ordem. Indique o valor inicial do semáforo e onde colocar as operações `wait` e `post`.

21. Uma conta de streaming permite, no máximo, quatro usuários simultâneos. Cada usuário é atendido por uma thread:

```
#define MAX_ACESSOS 4
int telas_em_uso = 0;

void* thread_usuario(void* arg) {
    int id = *((int*)arg);

    telas_em_uso++;
    acessar_conta(id);
    telas_em_uso--;
    return NULL;
}
```

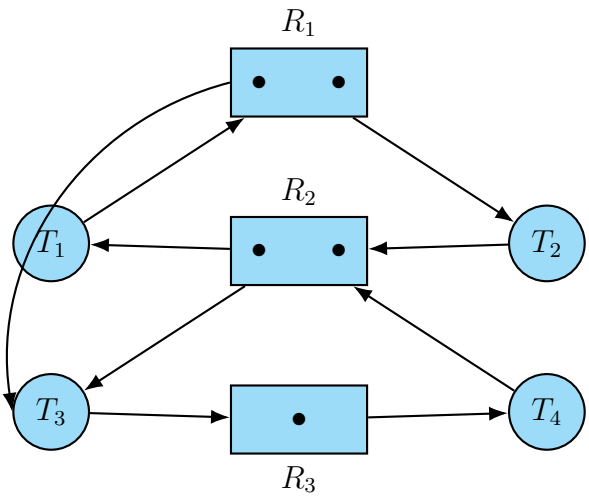
Use um semáforo contador para limitar o acesso a `MAX_ACESSOS` threads. Como `telas_em_uso` também é compartilhada, explique se outro mecanismo é necessário para protegê-la. Indique os valores iniciais e a posição de cada operação de sincronização.

5 Deadlocks

22. Descreva as quatro condições necessárias para que ocorra um travamento do sistema (*Deadlock*).

23. Apresente e explique uma estratégia técnica para prevenir a ocorrência de *Deadlocks*.

24. Dado o conjunto de threads $\{T_1, T_2, T_3, T_4\}$ e recursos $\{R_1, R_2, R_3\}$, analise se há deadlock no grafo abaixo:



Uma seta da thread para o recurso representa uma requisição. Uma seta do recurso para a thread representa uma instância alocada.

25. Considere dois semáforos, `S` e `Q`, ambos inicializados com o valor 1, e as duas threads abaixo:

```
1  void* thread1(void* arg) {
2      wait(S);
3      wait(Q);
4      usar_recursos();
5      post(Q);
6      post(S);
7      return NULL;
8  }
9
10 void* thread2(void* arg) {
11     wait(Q);
12     wait(S);
13     usar_recursos();
14     post(S);
15     post(Q);
16     return NULL;
17 }
```

Apresente uma ordem de execução das operações que causa deadlock e explique por que nenhuma thread consegue prosseguir. Em seguida, modifique a ordem de aquisição dos semáforos para impedir esse deadlock.

Soluções

Questão 1

O Sistema Operacional é um software que atua como intermediário entre o usuário e o hardware. Suas principais funções incluem: gerenciamento de processos (criação, escalonamento e término), gerenciamento de memória, gerenciamento de arquivos, gerenciamento de dispositivos de E/S e proteção/segurança do sistema.

Questão 2

A chamada de sistema (*system call*) é uma operação realizada pelo processo do usuário para requisitar serviços diretamente ao kernel do sistema operacional. É importante porque cria uma ponte controlada entre o espaço de usuário e o kernel, garantindo estabilidade e segurança ao isolar o hardware do acesso direto pelos processos.

Questão 3

Chamadas de sistema envolvem uma transição de modo (mudança de *user mode* para *kernel mode*) por meio de uma instrução específica do processador, sendo executadas com privilégios elevados. Chamadas de função comuns executam-se inteiramente no espaço do usuário, sem transição de modo e sem acesso direto a recursos protegidos do sistema.

Questão 4

Os estados de um processo são: Novo (criando), Pronto (esperando na fila para executar), Execução (instruções sendo executadas pela CPU), Espera/Bloqueado (aguardando algum evento como E/S) e Encerrado (terminou a execução).

Questão 5

O PCB (*Process Control Block*) é uma estrutura de dados do kernel que armazena todas as informações sobre um processo, incluindo: PID, estado do processo, contador de programa, registradores da CPU, informações de escalonamento, informações de memória (endereços base, limites), lista de arquivos abertos e informações de E/S.

Questão 6

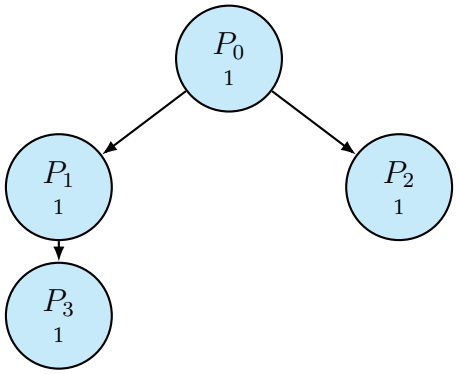
Processos possuem espaços de memória completamente isolados (cada um tem sua própria área de dados e código). Threads de um mesmo processo compartilham o mesmo espaço de endereço, ou seja, compartilham código, dados globais, heap e arquivos abertos, mas cada thread tem sua própria pilha e registradores.

Questão 7

a) A linha `Contador: 1` será impressa quatro vezes. O primeiro `fork()` cria um processo filho, totalizando dois processos. Ambos executam o segundo `fork()`, to-

talizando quatro processos. Cada processo possui sua própria cópia de `contador`, incrementa essa cópia de 0 para 1 e imprime uma linha. A ordem das quatro impressões não é determinada.

b) Uma possível identificação dos processos é mostrada abaixo. O número menor dentro de cada nó é o valor de `contador` no momento da impressão.

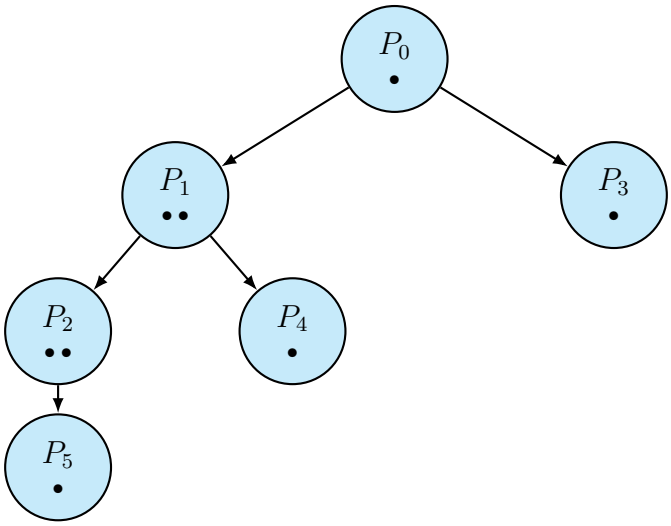


P_1 é criado pelo primeiro `fork()`. No segundo, P_0 cria P_2 e P_1 cria P_3.

Questão 8

a) A linha `Hello, World!` será impressa oito vezes. Depois do primeiro `fork()`, somente o filho entra no `if`. Esse filho cria mais um processo, e os dois imprimem dentro do `if`. Em seguida, os três processos existentes executam o último `fork()`, passando a ser seis; todos os seis executam a última chamada a `printf`. Portanto, ocorrem $2 + 6 = 8$ impressões.

b) Os pontos dentro de cada nó indicam quantas vezes aquele processo imprime a mensagem.



P_1 é criado pelo primeiro `fork()` e cria P_2 dentro do `if`. No último `fork()`, P_0, P_1 e P_2 criam, respectivamente, P_3, P_4 e P_5.

Questão 9

Sim, é vantajoso em muitos casos. Embora não exista paralelismo em um único núcleo, as threads podem melhorar a responsividade. Enquanto uma thread aguarda uma ope-

ração de E/S, o sistema operacional pode executar outra thread pronta, aproveitando melhor o processador.

Questão 10

O escalonador é o componente do kernel responsável por decidir qual processo da fila de prontos deve ser executado pela CPU. O custo da troca de contexto é o tempo gasto pelo processador para salvar o estado do processo que está saindo e carregar o estado do processo que está entrando. Esse tempo é sobrecarga, pois a CPU não executa instruções úteis do usuário durante a troca.

Questão 11

Escalonamento preemptivo permite que o sistema operacional interrompa um processo em execução a qualquer momento para ceder a CPU a outro processo (ex: algoritmos com prioridade ou quantum). No escalonamento não-preemptivo, o processo mantém a CPU até terminar sua execução ou bloquear-se voluntariamente para E/S.

Questão 12

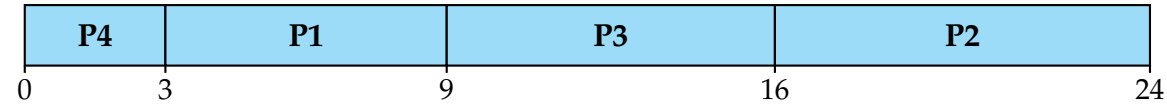
Um *quantum* muito pequeno aumenta significativamente o *overhead* de trocas de contexto (a CPU passa mais tempo trocando de processo do que executando). Um *quantum* muito grande piora o tempo de resposta para processos interativos (comportamento semelhante ao FIFO). O ideal é um equilíbrio onde o quantum seja significativamente maior que o tempo de troca de contexto.

Questão 13

Starvation (inanição) ocorre quando um processo nunca obtém os recursos necessários para concluir porque processos "preferenciais"sempre o antecedem. O SJF é suscetível porque processos longos podem ficar esperando indefinidamente se novos processos curtos chegarem constantemente, sendo sempre preteridos na escolha do próximo a executar.

Questão 14

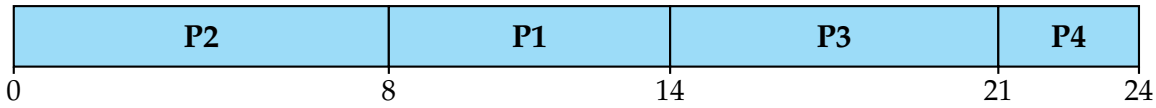
a) Shortest Job First (SJF):



Os tempos de espera são $T_{P4} = 0$, $T_{P1} = 3$, $T_{P3} = 9$ e $T_{P2} = 16$. Assim,

$$T_e = \frac{0 + 3 + 9 + 16}{4} = 7.$$

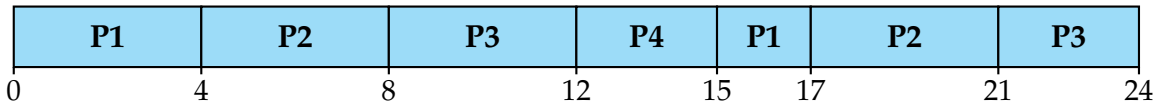
b) Prioridade:



Os tempos de espera são $T_{P2} = 0$, $T_{P1} = 8$, $T_{P3} = 14$ e $T_{P4} = 21$. Assim,

$$T_e = \frac{0 + 8 + 14 + 21}{4} = 10{,}75.$$

c) Round-Robin (q=4):



Como todos chegaram no tempo 0, o tempo de espera de cada processo é seu tempo de término menos seu pico de CPU. Logo,

$$T_{P1} = 17 - 6 = 11, \qquad T_{P2} = 21 - 8 = 13,$$
$$T_{P3} = 24 - 7 = 17, \qquad T_{P4} = 15 - 3 = 12.$$

Portanto,

$$T_e = \frac{11 + 13 + 17 + 12}{4} = 13{,}25.$$

Questão 15

Seção Crítica é a região do código que acessa variáveis ou recursos compartilhados e deve ser executada de forma atômica. Os três requisitos são: Exclusão Mútua (apenas uma thread na seção crítica por vez), Progresso (se nenhuma thread está na seção crítica, a escolha da próxima thread não pode ser adiada indefinidamente) e Espera Limitada (existe um limite para quantas vezes outras threads podem entrar depois que uma thread solicitou a entrada).

Questão 16

Mutex: É um mecanismo de exclusão mútua que possui o conceito de **dono** (somente a thread que fez o `lock` deve fazer o `unlock`). Semáforo contador: É um contador inteiro que pode gerenciar múltiplas instâncias de um recurso (por exemplo, cinco conexões de banco). Uma thread pode executar `wait` e outra pode executar `signal`; não há o mesmo conceito de dono.

Questão 17

Duas threads tentam incrementar uma variável `x` simultaneamente. Cenário: `x = 10`. A thread A lê 10. A thread B também lê 10. A calcula 11 e escreve esse valor. B também calcula 11 e o escreve. O resultado final é 11, embora o correto fosse 12. Um incremento foi perdido devido ao acesso concorrente sem sincronização.

Questão 18

A seção crítica é a instrução `saldo = saldo + valor;`. Duas threads podem ler o mesmo saldo, calcular resultados separadamente e fazer com que uma atualização sobrescreva a outra. O mutex deve envolver essa operação completa:

```
lock(mutex_saldo);
saldo = saldo + valor;
unlock(mutex_saldo);
```

A leitura de `arg` usa um argumento próprio da thread e pode ficar fora da seção crítica. A mensagem também pode ser impressa depois do `unlock`, pois não consulta `saldo`.

Questão 19

O teste e as duas atualizações dependem do mesmo estado e precisam formar uma única seção crítica. Proteger apenas as atribuições não basta, pois duas threads ainda poderiam aprovar suas compras antes de qualquer uma reduzir o estoque.

```
lock(mutex_estoque);
if (estoque >= quantidade) {
    estoque = estoque - quantidade;
    vendas = vendas + quantidade;
}
unlock(mutex_estoque);
```

Questão 20

O semáforo `relatorio_pronto` deve começar em 0. A thread `enviar` executa `wait` antes de usar o relatório e fica bloqueada caso ele ainda não esteja pronto. A thread `preparar` executa `post` somente depois de terminar a preparação:

```
void* preparar(void* arg) {
    preparar_relatorio();
    post(relatorio_pronto);
    return NULL;
}

void* enviar(void* arg) {
    wait(relatorio_pronto);
    enviar_relatorio();
    return NULL;
}
```

Questão 21

É necessário um semáforo contador `vagas`, inicializado com 4. Cada thread executa `wait(vagas)` antes de acessar a conta e `post(vagas)` ao terminar. Um mutex separado, inicializado como livre, protege cada alteração de `telas_em_uso`:

```
wait(vagas);

lock(mutex_telas);
telas_em_uso++;
unlock(mutex_telas);

acessar_conta(id);

lock(mutex_telas);
telas_em_uso--;
unlock(mutex_telas);

post(vagas);
```

O mutex não deve permanecer travado durante `acessar_conta`, pois isso reduziria o limite prático de quatro usuários para apenas um. Da mesma forma, um mutex sozinho não representa quatro unidades disponíveis; essa é a função do semáforo contador.

Questão 22

As quatro condições necessárias são: 1) Exclusão Mútua (recurso não pode ser compartilhado), 2) Posse e Espera (processo segura pelo menos um recurso e espera por outros), 3) Sem Preempção (recurso só é liberado voluntariamente) e 4) Espera Circular (existe uma cadeia de processos em que cada um espera por um recurso mantido pelo próximo).

Questão 23

Uma estratégia é **impor uma ordem de aquisição de recursos**. Se todos os processos forem obrigados a adquirir os recursos na mesma ordem (por exemplo, R_1 antes de R_2), a condição de espera circular não pode ocorrer. Assim, uma das quatro condições necessárias ao deadlock é eliminada.

Questão 24

Há deadlock. A tabela resume a situação mostrada no grafo:

Thread	Recursos que possui	Recurso solicitado
T_1	uma instância de R_2	R_1
T_2	uma instância de R_1	R_2
T_3	uma instância de R_1 e uma de R_2	R_3
T_4	uma instância de R_3	R_2

Todas as instâncias dos recursos estão alocadas, e nenhuma thread consegue prosseguir para liberar o que possui. Por exemplo, existe o ciclo $T_1 \to R_1 \to T_2 \to R_2 \to T_1$. Como R_1 e R_2 têm duas instâncias, observar apenas esse ciclo não seria suficiente. Neste caso, porém, as outras instâncias pertencem a T_3, que espera por R_3, mantido por T_4; por sua vez, T_4 espera por R_2. Assim, todo o conjunto permanece bloqueado.

Questão 25

Uma ordem de execução que causa deadlock é:

1) `thread1` executa `wait(S)` e obtém `S`;
2) `thread2` executa `wait(Q)` e obtém `Q`;
3) `thread1` executa `wait(Q)` e bloqueia;
4) `thread2` executa `wait(S)` e bloqueia.

Cada thread possui um semáforo e espera pelo semáforo mantido pela outra. Como nenhuma chega às operações `post`, ambas permanecem bloqueadas.

Para impedir a espera circular, as duas threads devem adquirir os semáforos na mesma ordem. Por exemplo, a segunda thread pode ser alterada para:

```
void* thread2(void* arg) {
    wait(S);
    wait(Q);
    usar_recursos();
    post(Q);
    post(S);
    return NULL;
}
```

Agora, se uma thread já possui `S`, a outra bloqueia antes de obter `Q`. A primeira pode obter os dois semáforos, terminar e liberá-los.